

Android 网络请求与数据解析

Android 网络请求+数据解析（Get/Post/图片上传）+第三方库详解

本文用 **最简洁的代码和说明**，覆盖 Android 基础网络请求（Get/Post/图片上传）、数据解析（JSON），以及常用第三方库的使用，适合快速入门。

一、核心前提：权限配置

所有网络操作都需要在 `AndroidManifest.xml` 中添加权限（Android 9.0+ 需额外配置明文网络）：

Code block

```
1  <!-- 网络权限 -->
2  <uses-permission android:name="android.permission.INTERNET" />
3  <!-- 读写权限（图片上传时需要，Android 6.0+ 需动态申请） -->
4  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
5
6  <!-- Android 9.0+ 允许明文网络（http），若用 https 可省略 -->
7  <application
8      ...
9      android:usesCleartextTraffic="true">
10     ...
11 </application>
```

二、原生网络请求（HttpURLConnection）

Android 原生网络请求核心类是 `HttpURLConnection`，**必须在子线程执行**（主线程禁止网络操作）。

1. Get 请求（获取数据）

功能：向服务器请求数据（参数拼在 URL 后）

步骤：

1. 子线程中创建 URL 对象；
2. 打开连接，设置请求方法为 GET；
3. 读取服务器响应数据；

4. 切换到主线程更新 UI。

代码：

Code block

```
1  // 1. 开启子线程 (用 Thread 简化, 实际推荐 AsyncTask/线程池)
2  new Thread(() -> {
3      HttpURLConnection conn = null;
4      BufferedReader reader = null;
5      try {
6          // 接口地址 (带参数: name=Android&age=10)
7          URL url = new URL("http://xxx.xxx.xxx/api/getData?
name=Android&age=10");
8          conn = (HttpURLConnection) url.openConnection();
9
10         // 配置请求: GET 方法、超时时间
11         conn.setRequestMethod("GET");
12         conn.setConnectTimeout(5000); // 连接超时 5s
13         conn.setReadTimeout(5000);    // 读取超时 5s
14
15         // 2. 发送请求, 获取响应码 (200 = 成功)
16         if (conn.getResponseCode() == HttpURLConnection.HTTP_OK) {
17             // 3. 读取响应数据 (服务器返回的 JSON 字符串)
18             reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
19             StringBuilder response = new StringBuilder();
20             String line;
21             while ((line = reader.readLine()) != null) {
22                 response.append(line);
23             }
24             // 4. 切换到主线程更新 UI (用 runOnUiThread)
25             runOnUiThread(() -> {
26                 Toast.makeText(MainActivity.this, "Get 成功: " +
response.toString(), Toast.LENGTH_SHORT).show();
27                 // 后续: 解析 JSON 数据 (见下文)
28             });
29         }
30     } catch (Exception e) {
31         e.printStackTrace();
32         runOnUiThread(() -> Toast.makeText(MainActivity.this, "Get 失败",
Toast.LENGTH_SHORT).show());
33     } finally {
34         // 关闭连接和流
35         if (reader != null) try { reader.close(); } catch (Exception e) {}
36         if (conn != null) conn.disconnect();
37     }
```

```
38    }).start();
```

2. Post 请求（提交数据）

功能：向服务器提交数据（参数在请求体中，更安全）

步骤：

1. 子线程中创建 URL 对象；
2. 打开连接，设置请求方法为 POST，允许写入请求体；
3. 拼接参数，写入请求体；
4. 读取服务器响应。

代码：

Code block

```
1  new Thread(() -> {
2      HttpURLConnection conn = null;
3      OutputStream os = null;
4      BufferedReader reader = null;
5      try {
6          URL url = new URL("http://xxx.xxx.xxx/api/postData");
7          conn = (HttpURLConnection) url.openConnection();
8
9          // 配置 Post 请求
10         conn.setRequestMethod("POST");
11         conn.setConnectTimeout(5000);
12         conn.setReadTimeout(5000);
13         conn.setDoOutput(true); // 允许写入请求体 (Post 核心)
14         conn.setRequestProperty("Content-Type", "application/x-www-form-
urlencoded"); // 表单格式
15
16         // 3. 拼接 Post 参数 (key=value&key2=value2)
17         String params = "username=test&password=123456";
18         os = conn.getOutputStream();
19         os.write(params.getBytes(StandardCharsets.UTF_8)); // 写入请求体
20
21         // 4. 读取响应
22         if (conn.getResponseCode() == HttpURLConnection.HTTP_OK) {
23             reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
24             StringBuilder response = new StringBuilder();
25             String line;
26             while ((line = reader.readLine()) != null) {
```

```

27         response.append(line);
28     }
29     runOnUiThread(() -> Toast.makeText(MainActivity.this, "Post 成功: "
+ response.toString(), Toast.LENGTH_SHORT).show());
30 }
31 } catch (Exception e) {
32     e.printStackTrace();
33     runOnUiThread(() -> Toast.makeText(MainActivity.this, "Post 失败",
Toast.LENGTH_SHORT).show());
34 } finally {
35     if (os != null) try { os.close(); } catch (Exception e) {}
36     if (reader != null) try { reader.close(); } catch (Exception e) {}
37     if (conn != null) conn.disconnect();
38 }
39 }).start();

```

3. 图片上传 (Post + Multipart 格式)

功能：上传本地图片到服务器（需用 Multipart 格式，支持文件+普通参数）

核心：请求头 `Content-Type: multipart/form-data`，请求体包含文件二进制数据

代码：

Code block

```

1  new Thread(() -> {
2      HttpURLConnection conn = null;
3      OutputStream os = null;
4      FileInputStream fis = null;
5      try {
6          URL url = new URL("http://xxx.xxx.xxx/api/uploadImage");
7          conn = (HttpURLConnection) url.openConnection();
8
9          // 配置上传请求
10         conn.setRequestMethod("POST");
11         conn.setConnectTimeout(10000);
12         conn.setReadTimeout(10000);
13         conn.setDoOutput(true);
14         conn.setDoInput(true);
15
16         // 关键: Multipart 格式 (boundary 是分隔符, 自定义任意字符串)
17         String boundary = "====AndroidUploadBoundary====";
18         conn.setRequestProperty("Content-Type", "multipart/form-data;
boundary=" + boundary);

```

```

19
20     os = conn.getOutputStream();
21     // 1. 写入普通参数 (如 userId)
22     String userIdParam = "--" + boundary + "\r\n" +
23         "Content-Disposition: form-data; name=\"userId\"\r\n\r\n" +
24         "1001\r\n";
25     os.write(userIdParam.getBytes(StandardCharsets.UTF_8));
26
27     // 2. 写入图片文件 (文件参数名: image, 文件名: test.jpg)
28     File imageFile = new File("/sdcard/Pictures/test.jpg"); // 本地图片路径
    (需动态权限)
29     String imageHeader = "--" + boundary + "\r\n" +
30         "Content-Disposition: form-data; name=\"image\"; filename=\""
+ imageFile.getName() + "\"\r\n" +
31         "Content-Type: image/jpeg\r\n\r\n"; // 图片 MIME 类型
32     os.write(imageHeader.getBytes(StandardCharsets.UTF_8));
33
34     // 写入图片二进制数据
35     fis = new FileInputStream(imageFile);
36     byte[] buffer = new byte[1024];
37     int len;
38     while ((len = fis.read(buffer)) != -1) {
39         os.write(buffer, 0, len);
40     }
41     os.write("\r\n".getBytes(StandardCharsets.UTF_8)); // 换行
42
43     // 3. 结束符 (必须)
44     String end = "--" + boundary + "--\r\n";
45     os.write(end.getBytes(StandardCharsets.UTF_8));
46
47     // 4. 读取响应
48     if (conn.getResponseCode() == HttpURLConnection.HTTP_OK) {
49         BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
50         StringBuilder response = new StringBuilder();
51         String line;
52         while ((line = reader.readLine()) != null) {
53             response.append(line);
54         }
55         runOnUiThread(() -> Toast.makeText(MainActivity.this, "上传成功: "
+ response.toString(), Toast.LENGTH_SHORT).show());
56     }
57     } catch (Exception e) {
58         e.printStackTrace();
59         runOnUiThread(() -> Toast.makeText(MainActivity.this, "上传失败",
Toast.LENGTH_SHORT).show());
60     } finally {

```

```
61         if (fis != null) try { fis.close(); } catch (Exception e) {}
62         if (os != null) try { os.close(); } catch (Exception e) {}
63         if (conn != null) conn.disconnect();
64     }
65 }).start();
```

注意：

- Android 6.0+ 需动态申请 `READ_EXTERNAL_STORAGE` 权限（读取本地图片）；
- 图片路径需正确（如 `/sdcard/Pictures/test.jpg` 是外部存储路径）。

三、数据解析（JSON 格式）

服务器返回的响应通常是 JSON 格式，Android 原生支持 `JSONObject` 解析，简单场景够用。

示例：解析 JSON 数据

假设服务器返回以下 JSON：

Code block

```
1  {
2    "code": 200,
3    "msg": "success",
4    "data": {
5      "name": "Android",
6      "version": "14"
7    }
8  }
```

解析代码：

Code block

```
1  // 上文网络请求中获取的 response 字符串
2  String jsonResponse = "..."; // 服务器返回的 JSON 字符串
3  try {
4      // 1. 解析外层对象
5      JSONObject rootObj = new JSONObject(jsonResponse);
6      int code = rootObj.getInt("code");
7      String msg = rootObj.getString("msg");
8
9      // 2. 解析内层 data 对象
10     JSONObject dataObj = rootObj.getJSONObject("data");
11     String name = dataObj.getString("name");
12     String version = dataObj.getString("version");
```

```

13
14     // 3. 打印结果
15     Log.d("Parse", "code: " + code + ", msg: " + msg + ", name: " + name + ",
    version: " + version);
16 } catch (JSONException e) {
17     e.printStackTrace();
18 }

```

复杂场景：

如果返回 JSON 数组（如 `{"data": [{"name": "A"}, {"name": "B"}]}`），用 `JSONArray` 解析：

Code block

```

1  JSONArray dataArr = rootObj.getJSONArray("data");
2  for (int i = 0; i < dataArr.length(); i++) {
3      JSONObject item = dataArr.getJSONObject(i);
4      String name = item.getString("name");
5  }

```

四、第三方库（推荐，简化开发）

原生 `HttpURLConnection` 代码繁琐（如图片上传、请求拦截、超时处理），实际开发优先用第三方库，以下是最常用的 2 个：

1. Retrofit（最主流）

简介：

- 基于 OkHttp 封装的 RESTful 网络框架；
- 注解式 API（如 `@GET`、`@POST`），代码简洁；
- 支持 JSON 自动解析（配合 Gson）、文件上传/下载、拦截器等。

基本使用步骤：

1. 添加依赖（build.gradle）：

Code block

```

1  dependencies {
2      // Retrofit 核心
3      implementation 'com.squareup.retrofit2:retrofit:2.9.0'
4      // Gson 转换器（自动解析 JSON）
5      implementation 'com.squareup.retrofit2:converter-gson:2.9.0'

```

```
6 }
```

2. 定义 API 接口：

Code block

```
1 public interface ApiService {
2     // Get 请求 (参数用 @Query)
3     @GET("api/getData")
4     Call<BaseResponse<AppInfo>> getData(@Query("name") String name,
5     @Query("age") int age);
6
7     // Post 请求 (表单参数用 @Field)
8     @FormUrlEncoded
9     @POST("api/postData")
10    Call<BaseResponse<String>> postData(@Field("username") String username,
11    @Field("password") String password);
12
13    // 图片上传 (@Multipart + @Part)
14    @Multipart
15    @POST("api/uploadImage")
16    Call<BaseResponse<String>> uploadImage(
17        @Part("userId") RequestBody userId, // 普通参数
18        @Part MultipartBody.Part image // 图片文件
19    );
20 }
```

3. 发起请求（结合 Gson 自动解析）：

Code block

```
1 // 1. 创建 Retrofit 实例
2 Retrofit retrofit = new Retrofit.Builder()
3     .baseUrl("http://xxx.xxx.xxx/") // 基础 URL
4     .addConverterFactory(GsonConverterFactory.create()) // 自动解析 JSON
5     .build();
6
7 // 2. 获取 API 实例
8 ApiService apiService = retrofit.create(ApiService.class);
9
10 // 3. 发起 Get 请求
11 Call<BaseResponse<AppInfo>> call = apiService.getData("Android", 10);
12 call.enqueue(new Callback<BaseResponse<AppInfo>>() {
13     @Override
14     public void onResponse(Call<BaseResponse<AppInfo>> call,
15     Response<BaseResponse<AppInfo>> response) {
```



```

15         // 成功: response.body() 已自动解析为实体类
16         if (response.isSuccessful()) {
17             BaseResponse<AppInfo> data = response.body();
18             Toast.makeText(MainActivity.this, "Retrofit Get 成功: " +
data.getData().getName(), Toast.LENGTH_SHORT).show();
19         }
20     }
21
22     @Override
23     public void onFailure(Call<BaseResponse<AppInfo>> call, Throwable t) {
24         // 失败
25         Toast.makeText(MainActivity.this, "Retrofit Get 失败",
Toast.LENGTH_SHORT).show();
26     }
27 });

```

核心优势：

- 无需手动处理子线程（`enqueue` 自动异步）；
- JSON 自动解析为实体类（避免 `JSONObject` 繁琐代码）；
- 图片上传、请求头配置等功能简化。

2. OkHttp（轻量级，Retrofit 底层依赖）

简介：

- Square 公司开发的轻量级网络库；
- 比 `HttpURLConnection` 更简洁，支持拦截器、缓存、WebSocket 等；
- 适合简单网络请求场景。

基本使用（Get 请求）：

1. 添加依赖：

Code block

```
1 implementation 'com.squareup.okhttp3:okhttp:4.11.0'
```

2. 发起请求：

Code block

```

1 // 1. 创建 OkHttpClient 实例
2 OkHttpClient client = new OkHttpClient.Builder()

```

```
3         .connectTimeout(5, TimeUnit.SECONDS)
4         .readTimeout(5, TimeUnit.SECONDS)
5         .build();
6
7 // 2. 构建 Get 请求
8 Request request = new Request.Builder()
9     .url("http://xxx.xxx.xxx/api/getData?name=Android&age=10")
10    .get() // 默认就是 Get, 可省略
11    .build();
12
13 // 3. 发起异步请求
14 client.newCall(request).enqueue(new Callback() {
15     @Override
16     public void onFailure(Call call, IOException e) {
17         // 失败 (子线程)
18         runOnUiThread(() -> Toast.makeText(MainActivity.this, "OkHttp Get 失败", Toast.LENGTH_SHORT).show());
19     }
20
21     @Override
22     public void onResponse(Call call, Response response) throws IOException {
23         // 成功 (子线程)
24         String responseStr = response.body().string();
25         runOnUiThread(() -> Toast.makeText(MainActivity.this, "OkHttp Get 成功: " + responseStr, Toast.LENGTH_SHORT).show());
26     }
27 });
```

五、总结

方案	优点	缺点	适用场景
原生 HttpURLConnection	无依赖，系统自带	代码繁琐，需手动处理线程、异常、解析	学习/简单Demo
Retrofit	代码简洁，自动解析，功能强大	需添加依赖，有一定学习成本	实际项目（主流选
OkHttp	轻量，灵活，支持高级功能（拦截器）	需手动解析 JSON，比 Retrofit 繁琐	简单请求/自定义需

开发建议：

1. 学习阶段：用原生 `HttpURLConnection` 理解网络请求原理；

2. 实际项目：优先用 `Retrofit + Gson`（高效、易维护）；
 3. 数据解析：简单场景用原生 `JSONObject`，复杂场景用 `Gson`（自动映射实体类）。
- （注：文档部分内容可能由 AI 生成）